

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу
«Операционные системы»

Группа: М8О-216Б-23

Студент: Ермакова М. П.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 10.10.24

Москва, 2024

Постановка задачи

Вариант 10.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода.

Стандартный поток вывода дочернего процесса перенаправляется в pipe1. Родительский процесс читает из pipe1 и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

Нужно взять свою первую лабу и переделать её с использованием shared memory и memory mapping.

10 вариант) В файле записаны команды вида: «число<newline>». Дочерний процесс производит проверку этого числа на простоту. Если число составное, то дочерний процесс пишет это число в стандартный поток вывода. Если число отрицательное или простое, то тогда дочерний и родительский процессы завершаются. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `int shm_open(const char *name, int oflag, mode_t mode)` – создание/открытие объекта разделяемой памяти
- `int ftruncate(int fd, off_t length)` – установка размера объекта разделяемой памяти
- `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset)` – отображение разделяемой памяти в адресное пространство процесса
- `sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value)` – создание/открытие именованного семафора
- `int sem_wait(sem_t *sem)` – ожидание семафора (уменьшение значения)
- `int sem_post(sem_t *sem)` – освобождение семафора (увеличение значения)
- `int munmap(void *addr, size_t length)` – удаление отображения разделяемой памяти
- `int shm_unlink(const char *name)` – удаление объекта разделяемой памяти
- `int sem_close(sem_t *sem)` – закрытие семафора
- `int sem_unlink(const char *name)` – удаление именованного семафора
- `pid_t wait(int *status)` – ожидание завершения дочернего процесса
- `int open(const char *pathname, int flags, mode_t mode)` – открытие файла
- `int close(int fd)` – закрытие файлового дескриптора
- `ssize_t read(int fd, void *buf, size_t count)` – чтение из файлового дескриптора
- `ssize_t write(int fd, const void *buf, size_t count)` – запись в файловый дескриптора
- `void exit(int status)` – завершение процесса

В рамках лабораторной работы была разработана Разработана многопроцессная система для обработки числовых данных из файла с использованием разделяемой памяти и семафоров. Система состоит из серверного и клиентского процессов, взаимодействующих через именованную разделяемую память.

Серверный процесс запрашивает имя файла, открывает его и создает объекты разделяемой памяти и семафоры для синхронизации. Затем он порождает клиентский процесс, который подключается к этим же объектам. Сервер последовательно читает числа из файла и записывает их в разделяемую память, а клиентский процесс считывает эти числа, проверяет их на простоту и выводит только непростые положительные числа. Встреча простого или отрицательного числа приводит к завершению клиента.

Взаимодействие процессов синхронизируется с помощью двух семафоров, обеспечивая корректный обмен данными. После завершения работы все ресурсы (разделяемая память и семафоры) корректно освобождаются серверным процессом.

Код программы

server.c

```
#include <stdbool.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/wait.h>

typedef struct {
    int number;
    bool finished;
    bool client_finished;
} shared_data;

int read_num_from_fd(int fd) {
    int number = 0;
    bool start = false;
    char c;
    size_t len;
    char flag = 1;

    while((len = read(fd, &c, 1)) > 0) {
        if (c == '-') {
            flag = -1;
        }
    }
}
```

```

    } else if (c >= '0' && c <= '9') {
        number = number * 10 + (c - '0');
        start = true;
    } else if (c == '\n' || c == ' ' || c == '\t' || c == '\r') {
        if (start) {
            return number * flag;
        }
    } else {
        const char mess[] = "Error: invalid character in input\n";
        write(STDERR_FILENO, mess, sizeof(mess) - 1);
        exit(EXIT_FAILURE);
    }
}

if (len == 0 && start) {
    return number * flag;
}
return -1;
}

int main() {
    char file_name[200];
    const char message[] = "Enter name of file: ";
    write(STDOUT_FILENO, message, sizeof(message) - 1);

    size_t len_name = read(STDIN_FILENO, file_name, sizeof(file_name) - 1);
    if (len_name <= 0) {
        const char mess[] = "Error reading the file\n";
        write(STDOUT_FILENO, mess, sizeof(mess) - 1);
        exit(EXIT_FAILURE);
    }

    if (file_name[len_name - 1] == '\n') {
        file_name[len_name - 1] = '\0';
    } else {
        file_name[len_name] = '\0';
    }

    int file = open(file_name, O_RDONLY);
    if (file == -1) {
        const char mess[] = "Error opening file\n";
        write(STDOUT_FILENO, mess, sizeof(mess) - 1);
        exit(EXIT_FAILURE);
    }

    pid_t pid = getpid();

```

```

char shm_name[32];
char sem_parent_name[32];
char sem_child_name[32];

snprintf(shm_name, sizeof(shm_name), "/lab_shm_%d", pid);
snprintf(sem_parent_name, sizeof(sem_parent_name), "/sem_parent_%d", pid);
snprintf(sem_child_name, sizeof(sem_child_name), "/sem_child_%d", pid);

int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
if (shm_fd == -1) {
    const char mess[] = "Error create shared memory\n";
    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

if (ftruncate(shm_fd, sizeof(shared_data)) == -1) {
    const char mess[] = "Error set size shared memory\n";
    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

shared_data *data = mmap(NULL, sizeof(shared_data), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
if (data == MAP_FAILED) {
    const char mess[] = "Error mmap shared memory\n";
    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

data->number = 0;
data->finished = false;
data->client_finished = false;

sem_t *sem_parent = sem_open(sem_parent_name, O_CREAT, 0666, 1);
sem_t *sem_child = sem_open(sem_child_name, O_CREAT, 0666, 0);

if (sem_parent == SEM_FAILED || sem_child == SEM_FAILED) {
    const char mess[] = "Error create semaphore\n";
    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

const pid_t child = fork();
switch (child) {
    case -1: {
        const char msg[] = "error: failed to spawn new process\n";

```

```

        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    break;
case 0: {
    close(file);

    char pid_str[32];
    snprintf(pid_str, sizeof(pid_str), "%d", pid);

    execl("./client", "client", pid_str, NULL);

    const char mess[] = "Error executing client\n";
    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

default: {
    int number;

    while(1) {
        sem_wait(sem_parent);

        if (data->client_finished) {
            break;
        }

        number = read_num_from_fd(file);
        data->number = number;

        if (number == -1) {
            data->finished = true;
            sem_post(sem_child);
            break;
        }
        sem_post(sem_child);
    }

    close(file);
    wait(NULL);

    munmap(data, sizeof(shared_data));
    close(shm_fd);
    shm_unlink(shm_name);
    sem_close(sem_parent);
    sem_close(sem_child);
}

```

```

        sem_unlink(sem_parent_name);
        sem_unlink(sem_child_name);
    }
    break;
}
return 0;
}

```

client.c

```

#include <stdbool.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <string.h>
#include <sys/stat.h>

```

```

typedef struct {
    int number;
    bool finished;
    bool client_finished;
} shared_data;

```

```

bool is_prime(int num) {
    if (num < 2) return false;
    if (num == 2) return true;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) {
            return false;
        }
    }
    return true;
}

```

```

void write_num(int n) {
    char buffer[32];
    int len = snprintf(buffer, sizeof(buffer), "%d\n", n);
    write(STDOUT_FILENO, buffer, len);
}

```

```

int main(int argc, char *argv[]) {
    if (argc != 2) {
        const char mess[] = "Usage: client <parent_pid>\n";

```

```

    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

int parent_pid = atoi(argv[1]);

char shm_name[32];
char sem_parent_name[32];
char sem_child_name[32];

snprintf(shm_name, sizeof(shm_name), "/lab_shm_%d", parent_pid);
snprintf(sem_parent_name, sizeof(sem_parent_name), "/sem_parent_%d", parent_pid);
snprintf(sem_child_name, sizeof(sem_child_name), "/sem_child_%d", parent_pid);

int shm_fd = shm_open(shm_name, O_RDWR, 0666);
if (shm_fd == -1) {
    const char mess[] = "Error open shared memory\n";
    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

shared_data *data = mmap(NULL, sizeof(shared_data), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
if (data == MAP_FAILED) {
    const char mess[] = "Error mmap shared memory\n";
    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

sem_t *sem_parent = sem_open(sem_parent_name, 0);
sem_t *sem_child = sem_open(sem_child_name, 0);

if (sem_parent == SEM_FAILED || sem_child == SEM_FAILED) {
    const char mess[] = "Error open semaphore\n";
    write(STDERR_FILENO, mess, sizeof(mess) - 1);
    exit(EXIT_FAILURE);
}

while(1) {
    sem_wait(sem_child);

    if (data->finished) {
        sem_post(sem_parent);
        break;
    }
    if (is_prime(data->number) || data->number < 0) {

```



```
        data->client_finished = true;
        sem_post(sem_parent);
        break;
    }

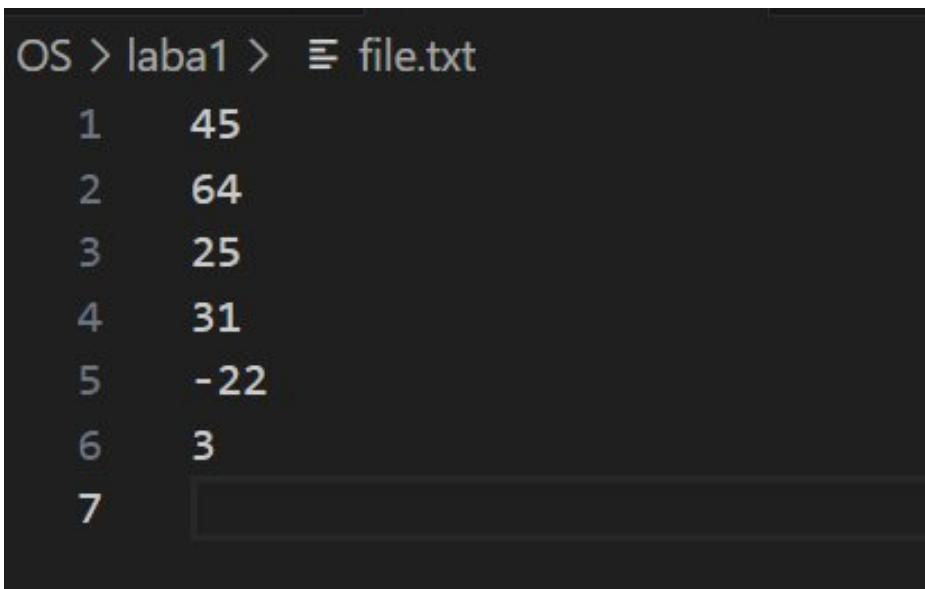
    write_num(data->number);
    sem_post(sem_parent);
}

munmap(data, sizeof(shared_data));
close(shm_fd);
sem_close(sem_parent);
sem_close(sem_child);

return 0;
}
```

Протокол работы программы

Содержимое файла file.txt:



```
OS > laba1 > file.txt
1    45
2    64
3    25
4    31
5    -22
6    3
7    
```

Вывод программы:

```
20  
many@ManyBook:/mnt/d/LAB2/OS/laba3$ strace -o strace.log ./server  
Enter name of file: file.txt  
45  
64  
25  
many@ManyBook:/mnt/d/LAB2/OS/laba3$ |
```

Тестирование:

\$./server

File.txt

Strace:

\$ strace -f -o trace.txt ./server

```
execve("./server", [".server"], 0x7ffe2fa1bc20 /* 27 vars */) = 0

brk(NULL) = 0x55d37c1c7000

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe35119a000

access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=19527, ...}) = 0

mmap(NULL, 19527, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe351195000

close(3) = 0

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784

fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784

mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fe350f83000

mmap(0x7fe350fab000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7fe350fab000

mmap(0x7fe351133000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7fe351133000

mmap(0x7fe351182000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7fe351182000

mmap(0x7fe351188000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fe351188000

close(3) = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe350f80000

arch_prctl(ARCH_SET_FS, 0x7fe350f80740) = 0

set_tid_address(0x7fe350f80a10) = 1579

set_robust_list(0x7fe350f80a20, 24) = 0

rseq(0x7fe350f81060, 0x20, 0, 0x53053053) = 0

mprotect(0x7fe351182000, 16384, PROT_READ) = 0

mprotect(0x55d35e5af000, 4096, PROT_READ) = 0

mprotect(0x7fe3511d2000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
```

[illegible]

mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 5, 0) = 0x7fe351197000

link("/dev/shm/sem.CAff2w", "/dev/shm/sem.sem_child_1579") = 0

fstat(5, {st_mode=S_IFREG|0644, st_size=32, ...}) = 0

unlink("/dev/shm/sem.CAff2w") = 0

close(5) = 0

clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7fe350f80a10) = 1580

read(3, "4", 1) = 1

read(3, "5", 1) = 1

read(3, "\r", 1) = 1

**futex(0x7fe351198000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY) = 0**

read(3, "\n", 1) = 1

read(3, "6", 1) = 1

read(3, "4", 1) = 1

read(3, "\r", 1) = 1

futex(0x7fe351197000, FUTEX_WAKE, 1) = 1

read(3, "\n", 1) = 1

read(3, "2", 1) = 1

read(3, "5", 1) = 1

read(3, "\r", 1) = 1

futex(0x7fe351197000, FUTEX_WAKE, 1) = 1

read(3, "\n", 1) = 1

read(3, "3", 1) = 1

read(3, "1", 1) = 1

read(3, "\r", 1) = 1

futex(0x7fe351197000, FUTEX_WAKE, 1) = 1

close(3) = 0

--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1580, si_uid=1000, si_status=0, si_etime=1 /*
0.01 s */ , si_stime=0} ---

wait4(-1, NULL, 0, NULL) = 1580

```
munmap(0x7fe351199000, 8)          = 0
close(4)                          = 0
unlink("/dev/shm/lab_shm_1579")   = 0
munmap(0x7fe351198000, 32)        = 0
munmap(0x7fe351197000, 32)        = 0
unlink("/dev/shm/sem.sem_parent_1579") = 0
unlink("/dev/shm/sem.sem_child_1579") = 0
exit_group(0)                     = ?
+++ exited with 0 +++
```

Вывод

Программа корректно создает дочерний процесс и организует передачу данных между процессами с использованием разделяемой памяти и семафоров, что подтверждается анализом системных вызовов strace.

Основные проблемы при выполнении работы возникли с пониманием принципов работы с разделяемой памятью, особенно в области правильного создания и управления именованными объектами shared memory, а также сложности вызвало обеспечение корректного доступа к одним и тем же объектам разделяемой памяти из разных процессов и правильное освобождение системных ресурсов (памяти и семафоров) при завершении работы программы.

В процессе работы были успешно реализованы уникальные имена для объектов разделяемой памяти и семафоров на основе PID процесса, что исключает конфликты при параллельном запуске нескольких экземпляров программы.